



<Name of Software Application>

CS 465 Project Software Design Document

Version 1.0

Table of Contents

CS 465 Project Software Design Document	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Design Constraints	3
System Architecture View	4
Component Diagram	4
Sequence Diagram	6
Class Diagram	8
API Endpoints	9
The User Interface	10
Editing Trip	10
Updated Trip	11
Reflection	11

Document Revision History

Version	Date	Author	Comments
1.3	08/15/26	Arden Boykin	Final Project

Executive Summary

Travlr Getaways needs a web application that handles a customer-facing website where travelers can browse and book trips, and an administrator single-page application (SPA) where staff can manage trip listings and reservations. The MEAN stack was used for development, which is made up of MongoDB, Express, Angular, and Node.js. This stack was used because all four components use JavaScript, so a single language can be used across the entire application rather than having to switch between different languages at different layers.

The customer-facing side is built with Express and Node.js on the server side. It uses the Handlebars (HBS) templating engine to render HTML pages dynamically from JSON data, and send them to the browser. The application follows an MVC pattern, meaning routes, controllers, and views are all kept separate, which will make the code easier to maintain and work with as the project grows. This approach works well for the public-facing site because pages load as complete HTML and don't depend on the browser having JavaScript enabled.

The administrator SPA is built with Angular and communicates with the back end through a RESTful API. Unlike the customer-facing site, the SPA loads once in the browser and updates content dynamically without reloading the whole page, which makes administrative tasks like adding or editing trips much faster. MongoDB is as the database for both sides of the application, providing flexible document-based storage that aligns well with the JSON data format used throughout the MEAN stack. Mongoose acts as the ODM layer between Express and MongoDB, providing schema definitions and a structured query interface.

Design Constraints

Since it is a web-based application, it has to run in a browser without any additional software installed on the client side. That limits what can be used on the front end to HTML, CSS, and JavaScript, and the application also needs to work well across different browsers and screen sizes.

Using the MEAN stack means everything has to be written in JavaScript. This is convenient in some ways but also means the project is tied to the Node.js ecosystem on the back end. Anyone working on the project needs to be comfortable with JavaScript in both a front-end and back-end context.

MongoDB is schema-less, which gives a lot of flexibility, but it also means the application itself has to handle data validation rather than relying on the database to do it. Mongoose is used to define schemas and enforce some structure, but that also adds an extra layer that has to be maintained.

The application must be able to handle multiple simultaneous connections without significant performance degradation, since a travel booking site could see bursts of traffic

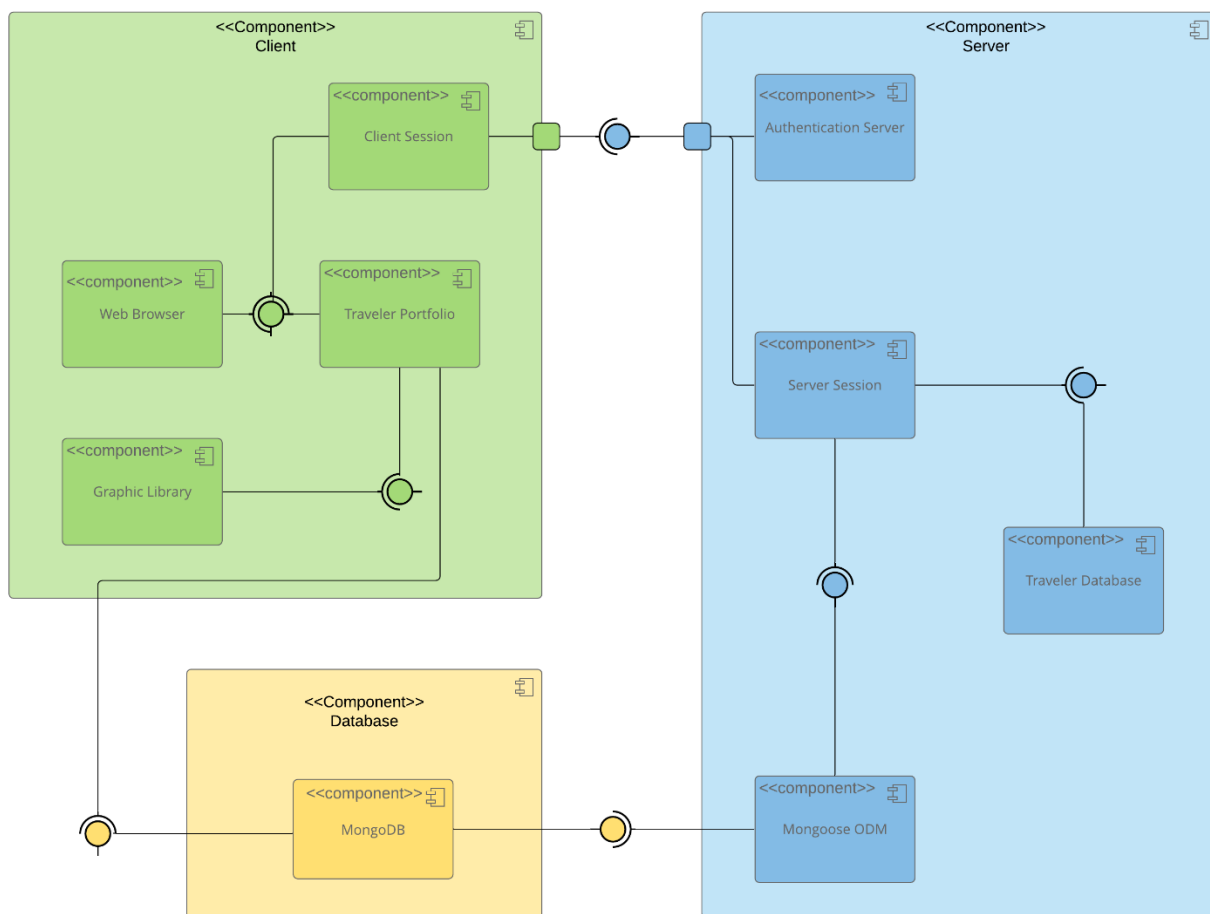
during promotions or peak travel seasons. This constrains architectural decisions around how the server handles requests and how the database is queried.

The admin SPA must be kept entirely separate from the customer-facing site in terms of access. This is a business requirement from the client that constrains how the application is structured, specifically that two distinct interfaces need to exist rather than combining admin and customer functionality into one.

The application must be deployable to a standard web server environment without requiring specialized infrastructure. This rules out certain architectural approaches that depend on proprietary platforms and reinforces the choice of a widely supported open-source stack like MEAN.

System Architecture View

Component Diagram



A text version of the component diagram is available: [CS 465 Full Stack Component Diagram Text Version](#).

The component diagram has a client layer, a server layer, and a data layer, each communicating with the one below it.

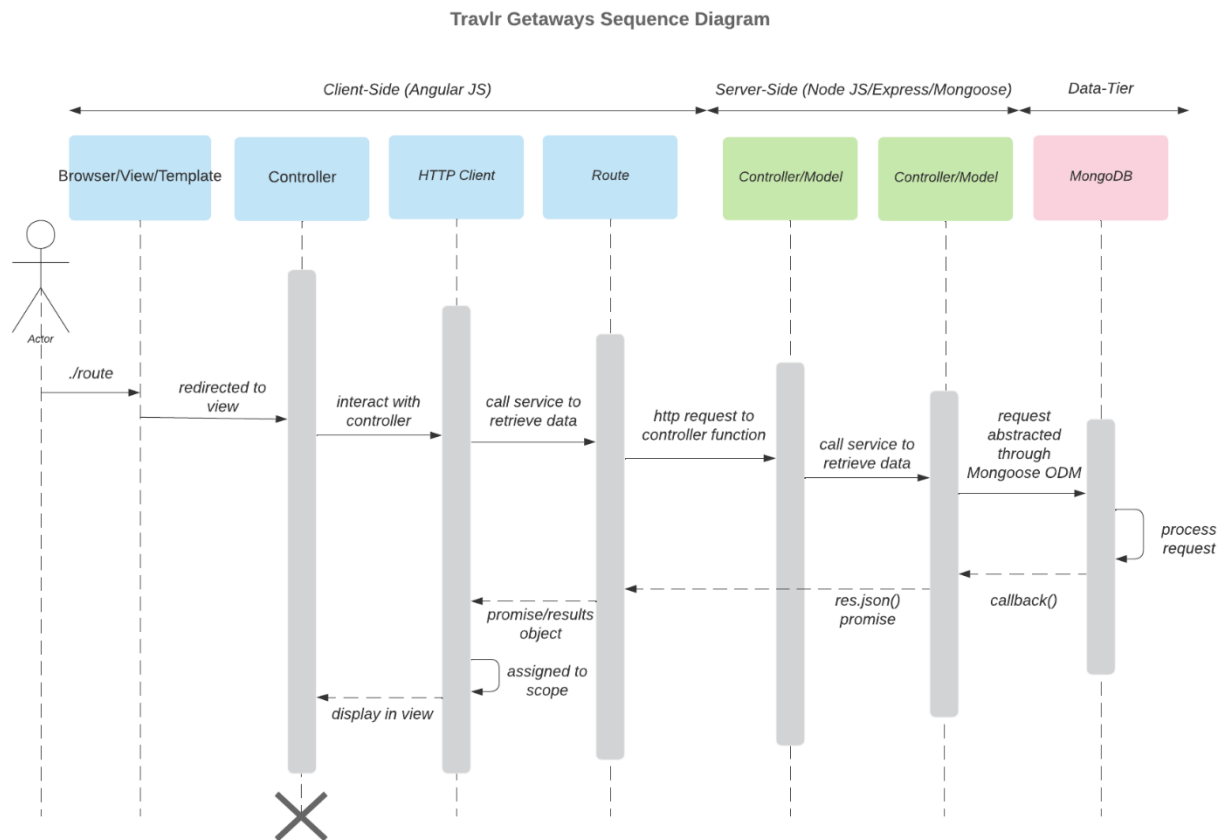
On the client side there are two separate interfaces. The first is the customer-facing browser, which receives fully rendered HTML from the Express server and displays it. The second is the Angular admin SPA, which runs in the browser and communicates with the server by making HTTP requests to the REST API instead of loading new pages.

The server tier is where Express and Node.js handle incoming requests. When a customer request comes in, Express routes it through the appropriate controller, which pulls data and passes it to an HBS template to be rendered before sending back the HTML response. When a request comes from the admin SPA, Express handles it through the API layer, sending and receiving JSON instead of rendering any HTML.

At the data tier, MongoDB stores all the application data like trips and user accounts. Mongoose works between Express and MongoDB and provides a way to define what the data should look like and query it in a more structured way than working with MongoDB directly.

The customer-facing site and the admin SPA are separate interfaces but share the same back end and database. The customer site gets server-rendered pages while the SPA gets raw data through the API, but both are ultimately reading from and writing to the same MongoDB instance through Express.

Sequence Diagram



The sequence diagram shows how a request moves through all three tiers of the application from the moment a user interacts with the browser to when data is returned and displayed.

The process starts when the actor (user) navigates to a route (`./route`), which redirects them to the Browser/View/Template. From there the view passes the interaction to the client-side Controller, which figures out what data is needed and calls a service through the HTTP Client to make a request to the server side.

On the server side, the Route receives the HTTP request and passes it to the first Controller/Model, which handles the incoming request and calls the appropriate service. That hands off to the second Controller/Model, which abstracts the database query through the Mongoose ODM (meaning that, rather than writing raw MongoDB queries, the controller calls Mongoose methods like `find()` that handle the translation) and sends it to MongoDB. MongoDB processes the request and returns a `callback()` back to the Controller/Model layer, which

packages the result as a `res.json()` promise that travels back through the Route to the HTTP Client.

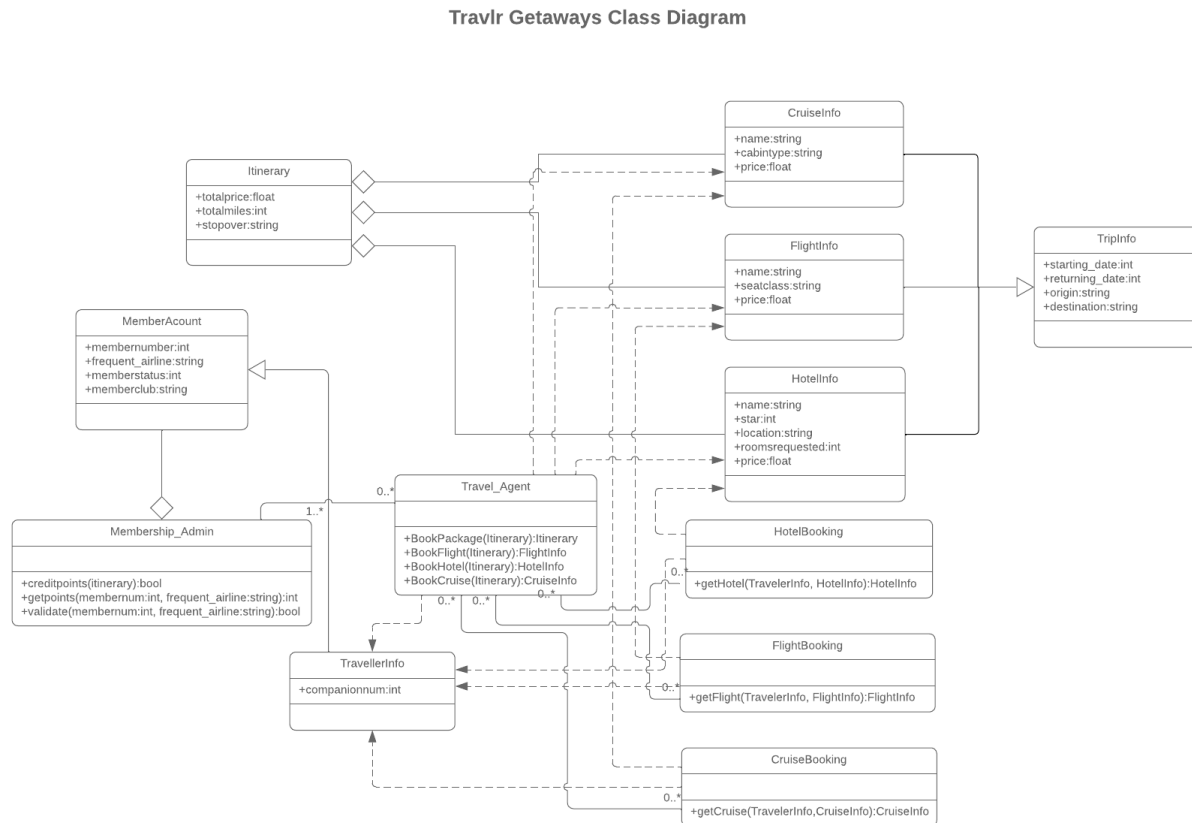
The HTTP Client receives the promise and passes the results object back to the client-side Controller, where the data gets assigned to scope. The Controller then passes the data to the Browser/View/Template to display to the user. The sequence ends at the view layer, shown by the X on the Browser/View/Template lifeline.

For the Trips interaction specifically, the customer navigates to `/travel`, the route hands it to the Express travel controller, which makes a fetch request to `/api/trips`. The API controller queries MongoDB through Mongoose and returns the full trips collection as JSON, which the travel controller passes to the HBS template for rendering.

For the Sign In interaction, the user submits credentials through the browser view, the Angular controller sends the credentials via HTTP Client to the authentication route on the server side, and the Controller/Model layer validates the credentials against the users collection in MongoDB. A successful validation returns a token that the Angular controller stores and uses for subsequent authenticated requests.

For Admin interactions, the Angular SPA controller makes authenticated HTTP requests through the HTTP Client to the appropriate API route. The server-side Controller/Model processes the request, performs the relevant MongoDB operation such as inserting or updating a trip, and returns the result as JSON. The Angular controller receives the response and updates the SPA view without a full page reload, which is the key behavioral difference between the admin SPA and the customer-facing site.

Class Diagram



The class diagram shows the JavaScript classes that make up the TravlR Getaways application and how they relate to each other.

Itinerary is the central class and acts as the main container for a trip, storing the total price, total miles, and any stopovers. It has aggregation relationships with **CruiseInfo**, **FlightInfo**, and **HotelInfo**, meaning a single itinerary can be made up of one or more of each type. **CruiseInfo** holds the cruise name, cabin type, and price. **FlightInfo** stores the seat class and price for a flight, and **TripInfo** holds the specific travel dates, origin, and destination that go along with it. **HotelInfo** stores the hotel name, star rating, location, rooms requested, and price.

Travel_Agent handles the actual booking logic and has methods for booking a full package or individual components through **BookPackage**, **BookFlight**, **BookHotel**, and **BookCruise**. Each of these takes an **Itinerary** as input and returns the relevant info object. **Travel_Agent** also connects to **TravellerInfo**, which represents the traveler and tracks how many companions are on the booking.

HotelBooking, **FlightBooking**, and **CruiseBooking** are three separate classes that each handle retrieving available options for their respective booking types. Each one has a single get method that takes **TravellerInfo** and the relevant info class as parameters and returns the

matching result. These are kept separate from Travel_Agent to distinguish between looking something up and actually booking it.

MemberAccount stores loyalty program data like member number, frequent airline, membership status, and member club. TravellerInfo extends MemberAccount through a generalization relationship, meaning a traveler inherits the membership data. Membership_Admin handles the loyalty program logic with methods to validate a member account, get a member's points balance, and credit points to an itinerary. The multiplicity notations in the diagram show that one Travel_Agent can handle many bookings across all three booking types, and that an Itinerary can be associated with multiple traveler records.

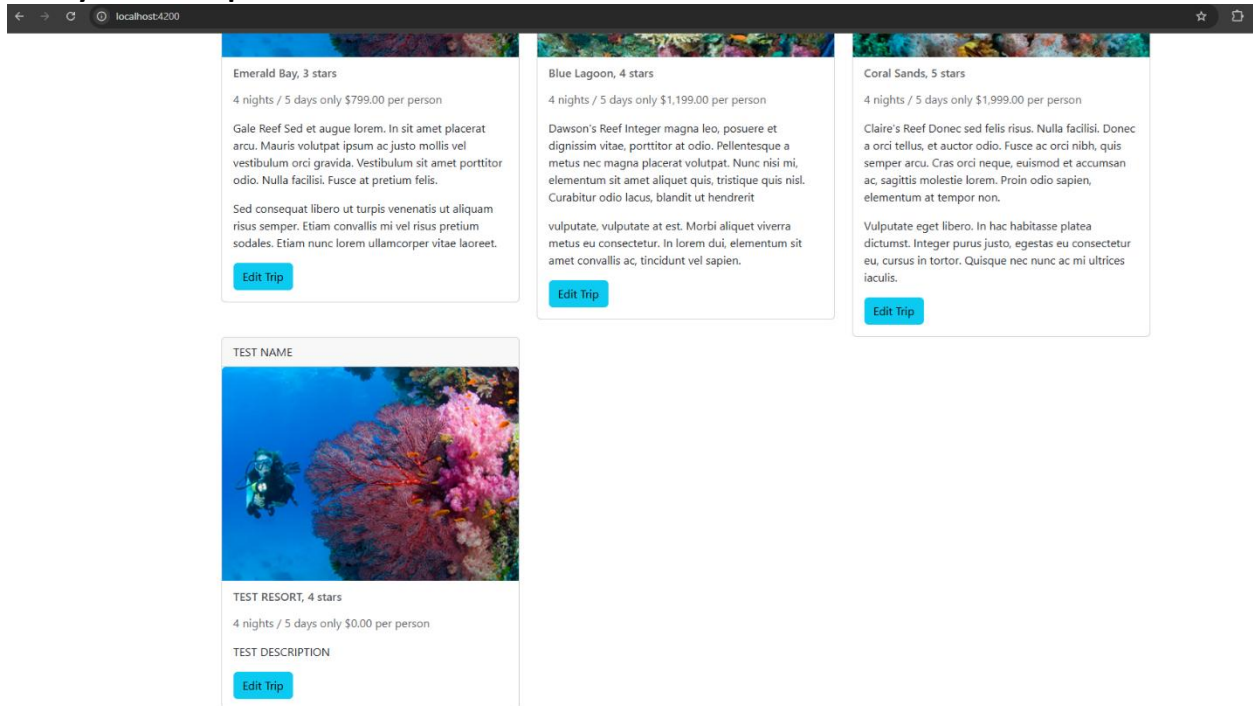
API Endpoints

Exposing RESTful endpoints is a design approach to enable an application to participate in a larger ecosystem. Each endpoint is documented in the table below, including the HTTP method, purpose, URL, and notes.

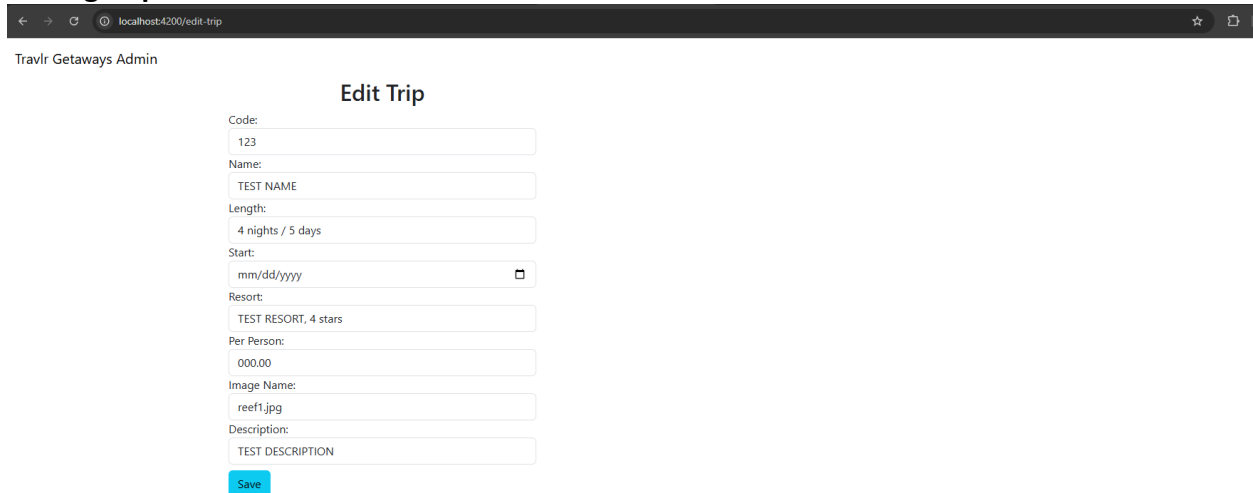
Method	Purpose	URL	Notes
GET	Retrieve list of all trips	/api/trips	Returns all trip documents from the MongoDB trips collection as a JSON array. Returns 404 if no data is found
GET	Retrieve a single trip by code	/api/trips/:tripCode	Returns a single trip document matching the trip code passed as a URL parameter. Returns 404 if no matching record exists
POST	<Add a new trip>	/api/trips	Returns a single trip document matching the trip code passed as a URL parameter. Returns 404 if no matching record exists
PUT	<Update an existing trip>	api/trips/:tripcode	Accepts a JSON body and updates the trip document matching the provided trip code. Returns 404 if the trip is not found

The User Interface

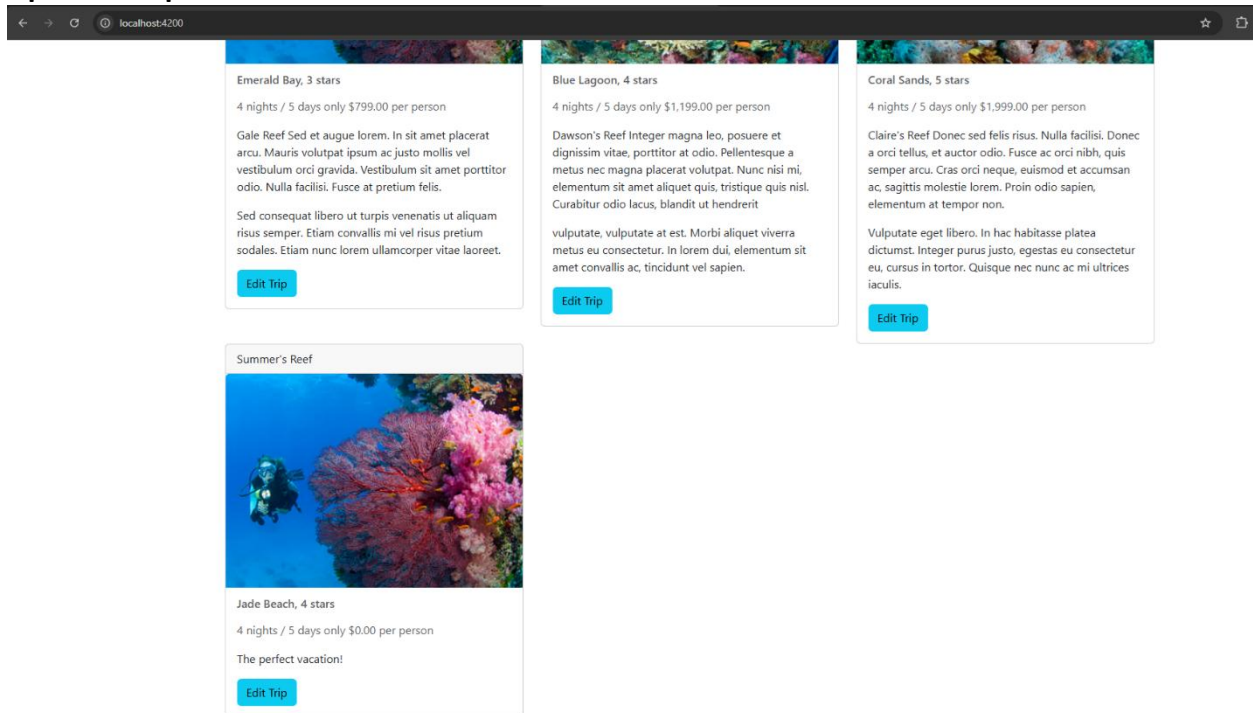
Newly Added Trip



Editing Trip



Updated Trip



Reflection

Building the Angular admin site made it clear pretty quickly that it's a fundamentally different approach to web development compared to what we did with Express. With Express, the server does most of the work: it receives a request, builds the HTML, and sends a finished page back to the browser. The browser just displays what it receives. With Angular, the server sends a single HTML shell on the first load and from that point on Angular takes over. It can swap components in and out without ever requesting a new page from the server. The project structure also reflects the difference. The Express side is organized around routes, views, and controllers where each URL corresponds to a template the server renders. The Angular side is organized around components, services, and a client-side router. Files like `app.routes.ts` and `app.config.ts` handle what `app.js` and the route files handle on the Express side, but all of it runs in the browser instead of on the server. Angular also uses TypeScript throughout, which introduces things like the Trip interface that have no real equivalent in the Express setup.

The biggest advantage of the SPA approach is responsiveness. Switching between the trip listing and the add or edit forms doesn't require a round trip to the server, so the transitions happen almost instantly. The component structure also keeps things organized in a way that's easy to maintain. The `TripDataService` handles all communication with the API, the `TripListingComponent` manages what gets displayed, and the `TripCardComponent` handles each individual card. Each part can be modified without touching the others. The downside is that there are a lot of moving parts to get right before anything works at all. The angular.json asset configuration, the CORS headers on the Express side, the HTTP client provider in `app.config.ts`,

and the reactive forms module imports all have to be correct or the application fails in ways that aren't always obvious. Compared to a basic web application, a SPA adds interactivity: forms that validate in real time, views that update without reloading, and components that maintain their own state while the user navigates around.

Testing the SPA required verifying two separate layers. Postman was the most useful tool for verifying the backend. Sending GET requests to `http://localhost:3000/api/trips` confirmed the backend was returning data correctly before touching the Angular side at all. Testing POST and PUT requests in Postman confirmed those endpoints worked before the forms were even built. Once the Angular app was running, the browser's network tab in the developer tools showed whether Angular's HTTP calls were actually reaching the server and what responses were coming back. Most of the errors I ran into were configuration problems rather than logic errors. The asset paths in `angular.json` used invalid syntax that prevented images from loading. The CORS setup in `app.js` initially only allowed GET and POST, which caused PUT requests from the edit form to fail without a clear error message. There was also a change detection issue where the trip listing wasn't updating visually even though the data was sitting in memory, which required manually triggering change detection after the API response came back. These kinds of errors were frustrating to debug because the problem wasn't always in the place the error message pointed to.

The biggest question I have is how authentication would fit into this architecture. The admin SPA is currently open to anyone who finds the URL, which clearly wouldn't work in a real deployment. I'm curious how Angular route guards work alongside something like JWT tokens on the Express backend.